

# 单细胞高维向量近似最近邻检索系统

A Web-based ANN Retrieval Platform for Single-cell Data

软件工程大作业 · 课程答辩

FastAPI

React 19

FAISS · HNSWLIB

RAG

团队 XXX · 2026 年 5 月

# 目录 Outline

| # | 章节    | 关键词                          |
|---|-------|------------------------------|
| 1 | 项目背景  | 单细胞测序 · ANN 必要性              |
| 2 | 需求分析  | 课程要求 · 功能矩阵 · 加分项            |
| 3 | 系统设计  | 架构 · 技术栈 · 引擎抽象 · 数据模型       |
| 4 | 核心实现  | 检索流水线 · 关键代码                 |
| 5 | 加分功能  | 多数据集联合 · Adaptive HNSW · RAG |
| 6 | 实测与交付 | 性能数据 · 演示截图 · 质量保障 · 总结      |

# 1. 项目背景 Background

## 单细胞测序 (scRNA-seq) 数据特征

- 一次实验可生成 十万级 细胞样本，每个细胞数值化为高维向量
- 本项目使用 CZI 公开数据 `liver.h5ad`：69 032 细胞 × 30 维 PCA 向量

## 痛点

- 精确 KNN 在  $N=10^5$ 、 $D=30\sim 512$  时延迟达数百毫秒，难以支持交互式查询
- 单细胞分析平台普遍依赖 Jupyter 脚本，缺乏面向 Web 的统一检索入口

## ANN 的价值

- 在召回率仅损失  $< 0.5\%$  的前提下，将单次查询从亚秒级压到 微秒级
- 配合元数据过滤、可视化、自然语言问答，可承载真实科研场景

## 2. 需求分析 Requirements

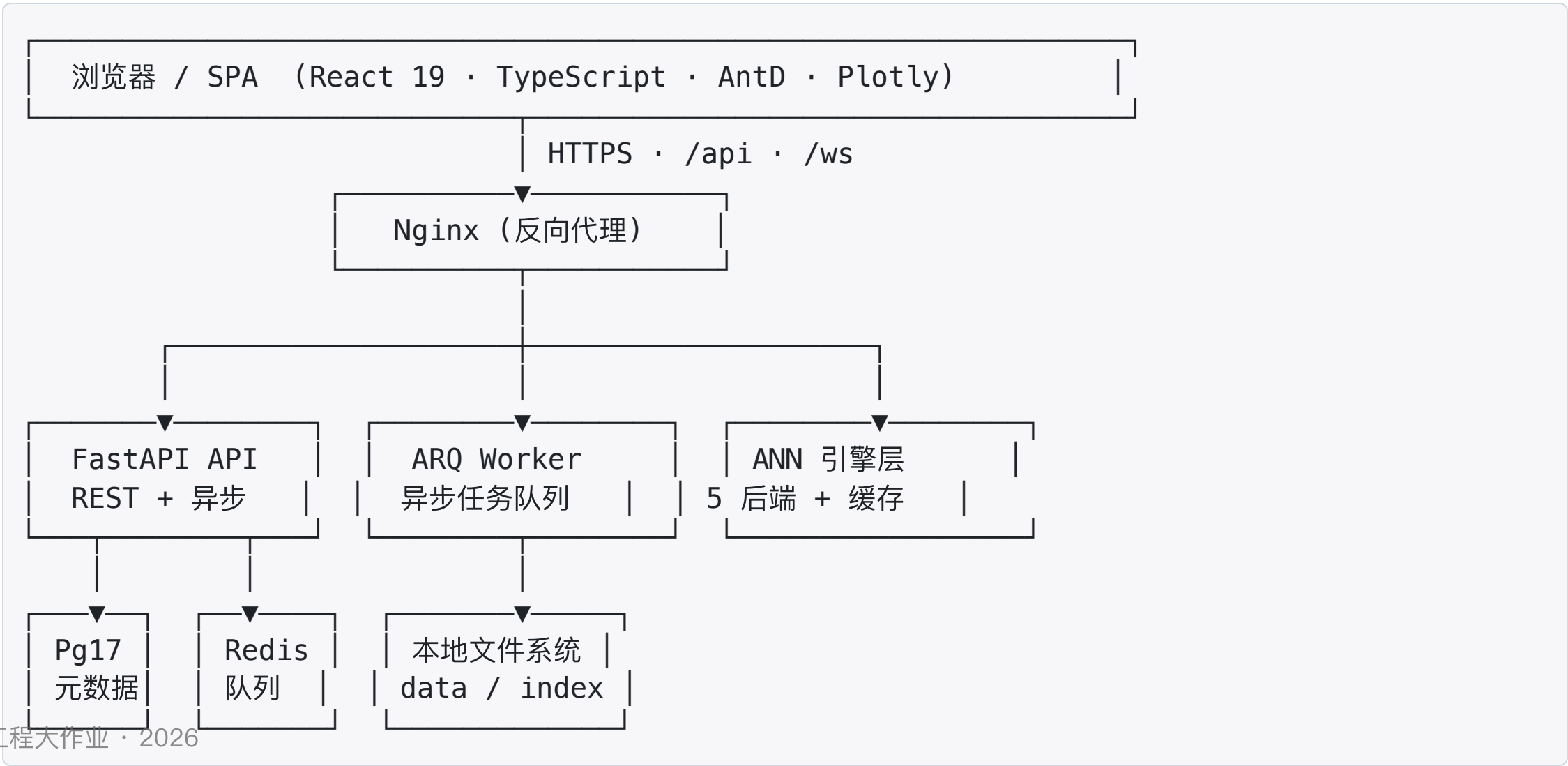
### 课程核心要求（必做）

| 模块   | 核心交付                           |
|------|--------------------------------|
| 用户信息 | 注册 / 登录 / JWT 鉴权 / 管理员角色       |
| 数据管理 | .h5ad 上传、Scanpy 预处理、向量提取       |
| 索引构建 | 多后端 / 可保存可加载 / 异步任务            |
| 查询检索 | 按 cell_id / 按向量 / 条件过滤 / Top-K |
| 可视化  | UMAP 投影、检索结果高亮、性能图表            |

### 加分功能（全做）

- 多数据集联合检索：并发 + min-max 归一化 + 重排
- ANN 算法改进：自适应 HNSW（自研后端）
- RAG + 单细胞：自然语言 → 解析 → 检索 → 总结

### 3. 系统架构总览 Architecture



# 3.1 技术栈 Tech Stack

| 层      | 选型                                                           | 用途                                       |
|--------|--------------------------------------------------------------|------------------------------------------|
| 前端     | React 19 · TypeScript · Vite · Ant Design · Zustand · Plotly | SPA · 状态管理 · 交互式可视化                      |
| 后端     | Python 3.12 · FastAPI · SQLAlchemy 2 async · Pydantic v2     | 异步 REST · ORM · 数据校验                     |
| 任务队列   | ARQ + Redis                                                  | 索引构建 / 预处理后台任务                           |
| 数据库    | PostgreSQL 17                                                | 用户 / 数据集 / 索引 / 检索日志                     |
| 缓存     | Redis 7                                                      | 查询缓存 + ARQ broker                        |
| ANN 引擎 | FAISS · HNSWLIB · scikit-learn                               | 5 种后端 (HNSW · IVF-PQ · Brute · Adaptive) |
| 单细胞    | scanpy · anndata · numpy · scipy                             | h5ad 读取 · PCA / UMAP                     |
| LLM    | DashScope · OpenAI 兼容 · Mock                                 | RAG 自然语言查询                               |
| 基础设施   | Docker Compose · Nginx · GitHub Actions · pre-Commit         | 部署 · CI · 代码规范                           |

# 3.2 ANN 引擎抽象 IndexBackend

## 统一接口

```
IndexBackend (ABC)
├─ name           — 后端标识
├─ build(X, **)   — 构建索引
├─ search(q, k)   — Top-K 检索
├─ save / load    — 持久化
└─ memory_mb()    — 内存估计
```

## 5 个具体后端 + 工厂 + 缓存

| 后端            | 实现库           | 特点                 |
|---------------|---------------|--------------------|
| brute         | numpy         | 精确，作为 ground truth |
| hnswlib       | hnswlib 0.8.0 | 高召回 + 微秒级延迟        |
| faiss-hnsw    | faiss 1.13    | OMP 多线程友好          |
| faiss-ivfpq   | faiss 1.13    | 量化压缩，内存极小          |
| adaptive-hnsw | 自研            | 早停 + 升档（加分项）       |

## 4. 核心代码：抽象 + 自适应 ef

### IndexBackend 抽象

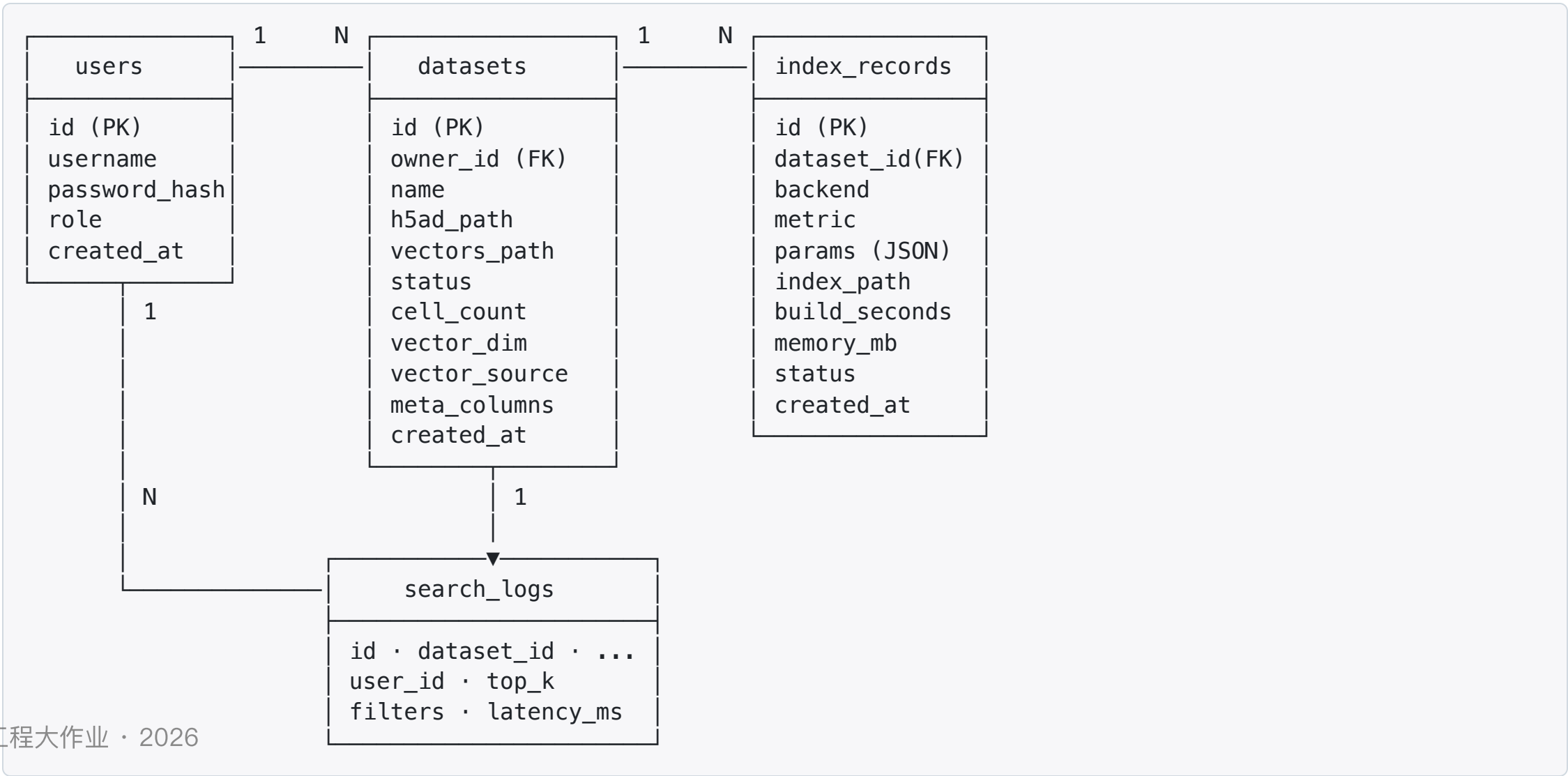
```
class IndexBackend(ABC):
    @property
    @abstractmethod
    def name(self) -> str: ...
    @abstractmethod
    def build(self, vectors: np.ndarray, **params) -> None: ...
    @abstractmethod
    def search(self, query: np.ndarray, top_k: int) -> tuple[np.ndarray, np.ndarray]: ...
    @abstractmethod
    def save(self, path: str) -> None: ...
    @abstractmethod
    def load(self, path: str) -> None: ...
    @abstractmethod
    def memory_mb(self) -> float: ...
```

### 自适应 HNSW 核心循环（节选）

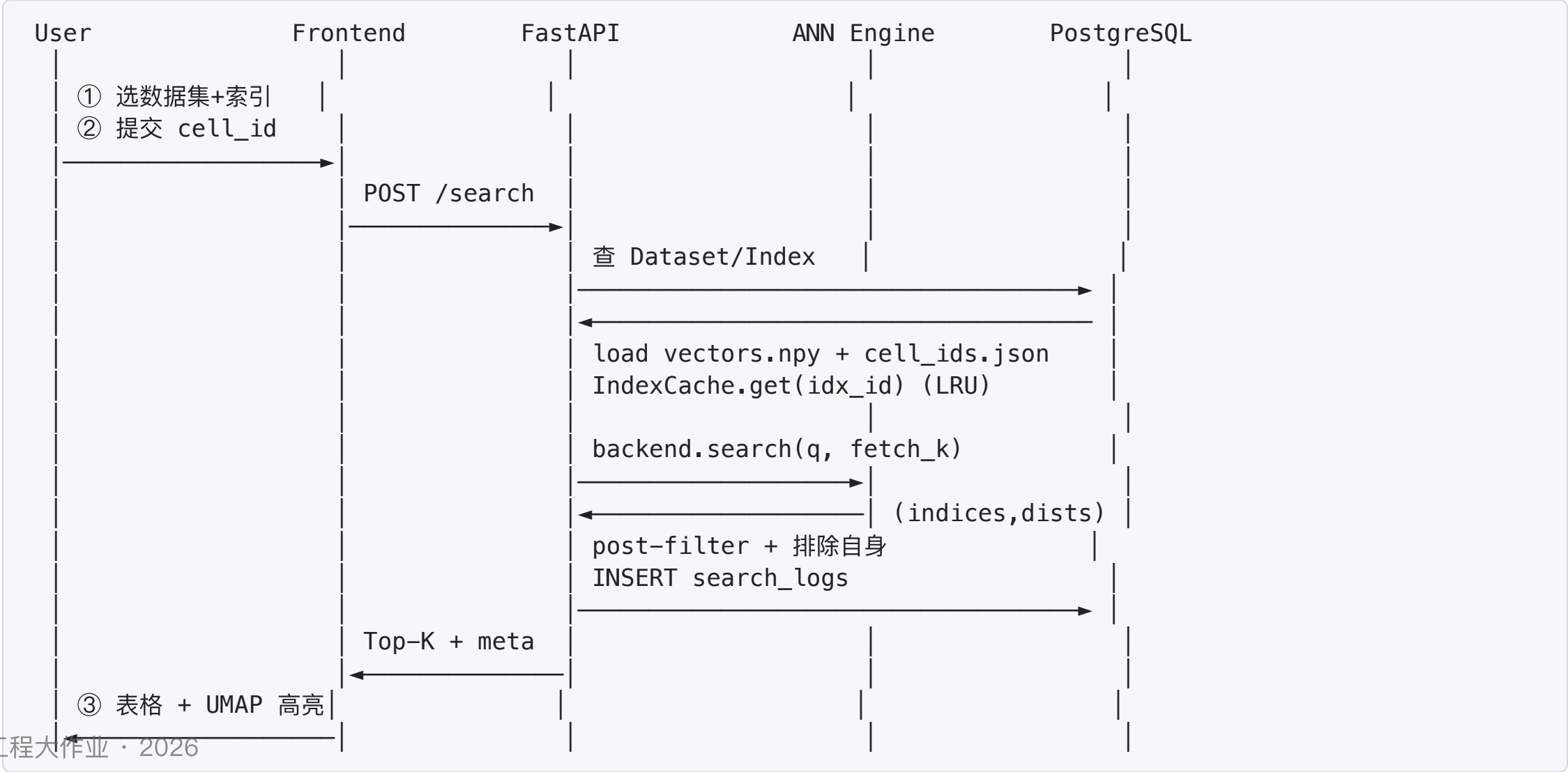
```
while pending.size > 0:
    self._index.set_ef(int(max(ef, k_query)))
    labels, dists = self._index.knn_query(q[pending], k=k_query)
    if retry == 0: # 首轮：相对距离间隔早停
```



# 4.1 数据模型 ER



## 4.2 检索流水线时序



## 5. 加分项 ①：多数据集联合检索

问题：用户希望"在多个肝脏数据集里一起找相似细胞"

### 实现要点

1. 路由层 `POST /search/multi-dataset`，接收 `dataset_ids: [int]`
2. 对每个数据集 **并发** 调用 `async_search_by_vector` (`asyncio.gather`)
3. 每个数据集结果做 **min-max 归一化**：`norm = (d - dmin) / (dmax - dmin)`
4. 合并后按 `normalized_distance` 升序，统一 Top-K，并标注 `source_dataset_id`

```
merged.sort(key=lambda x: x["normalized_distance"])
final = merged[:top_k]
for i, item in enumerate(final, start=1):
    item["rank"] = i
```

**收益：**跨数据集语义可比；并发  $\text{latency} \approx \max(\text{per\_dataset})$ ，而非求和。

# 5. 加分项 ②：自适应 HNSW

动机：固定 `ef_search` 在 query 难度分布不均时浪费算力 / 召回不足。

策略（继承 `HnswlibBackend`，仅重写 `search`）

- 起始 `ef = min_ef (32)`，oversample 多取 8 个候选
- 首轮判稳：相对距离间隔  $gap = (d[k] - d[k-1]) / (d[k-1] - d[0])$ ，越大说明 Top-K 边界越清晰
- 后续轮判稳：与上一轮 Top-K 的集合重合度 `overlap@k`
- 升档：`ef = min(ef * 2, max_ef=512)`；按 query 粒度独立判定，已稳定提前返回

实测 (top\_k=10, N=30 000)

| 指标             | hnswlib 固定 ef=64 | adaptive-hnsw |
|----------------|------------------|---------------|
| p50 / p95 (ms) | 0.016 / 0.020    | 0.045 / 0.057 |
| Recall@10      | 0.9996           | 0.9994        |
| mean_ef        | 64               | 50.6          |

## 5. 加分项 ③：RAG 自然语言查询

流程： `parse_query` → ANN search → `summarize`



三客户端协议化设计 —— 无外网也可演示

## 6. 实测数据：liver.h5ad

数据集：CZI 儿童肝脏 scRNA-seq, 69 032 细胞 × 30 维 PCA (基准 N=30 000)

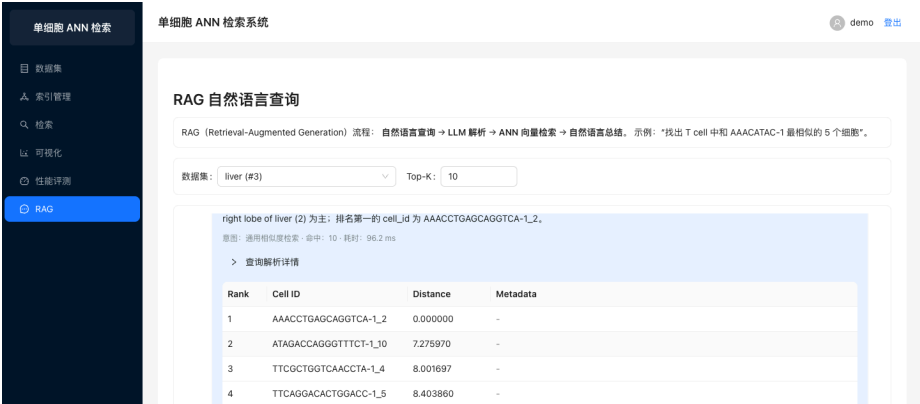
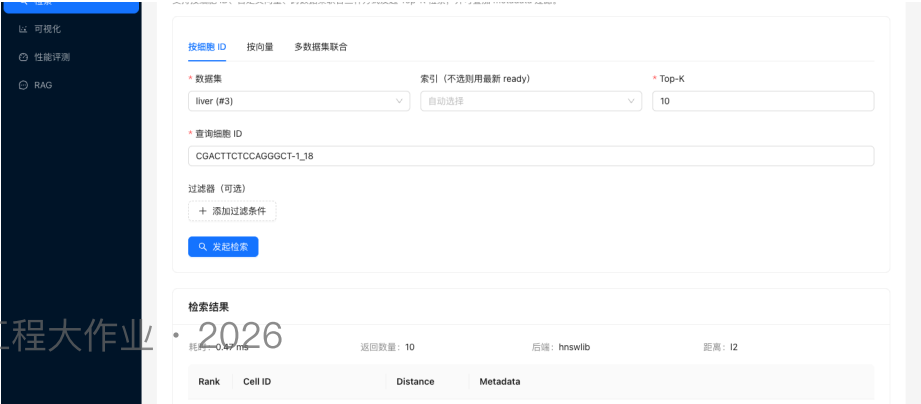
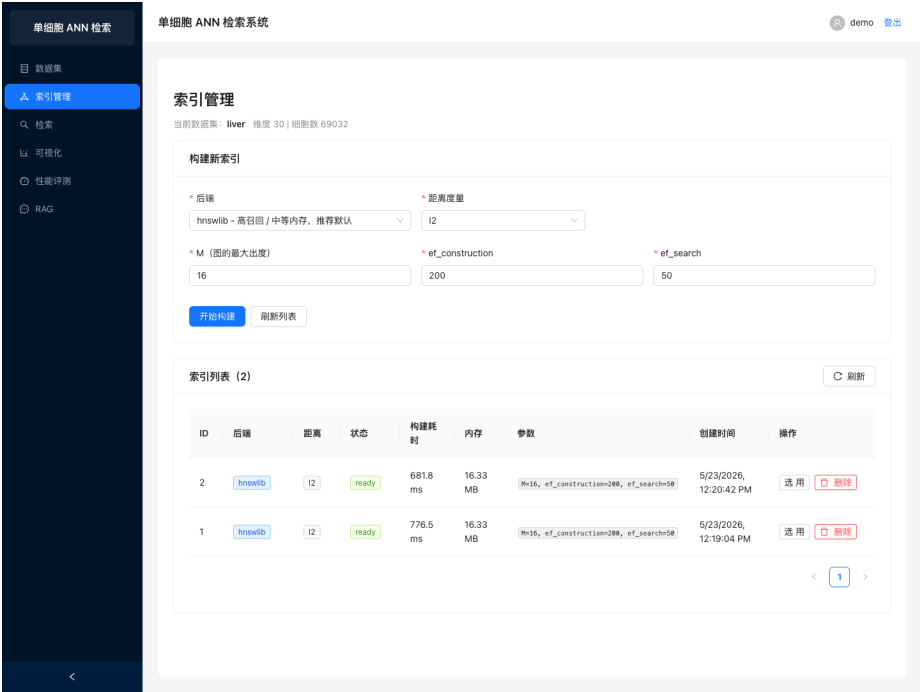
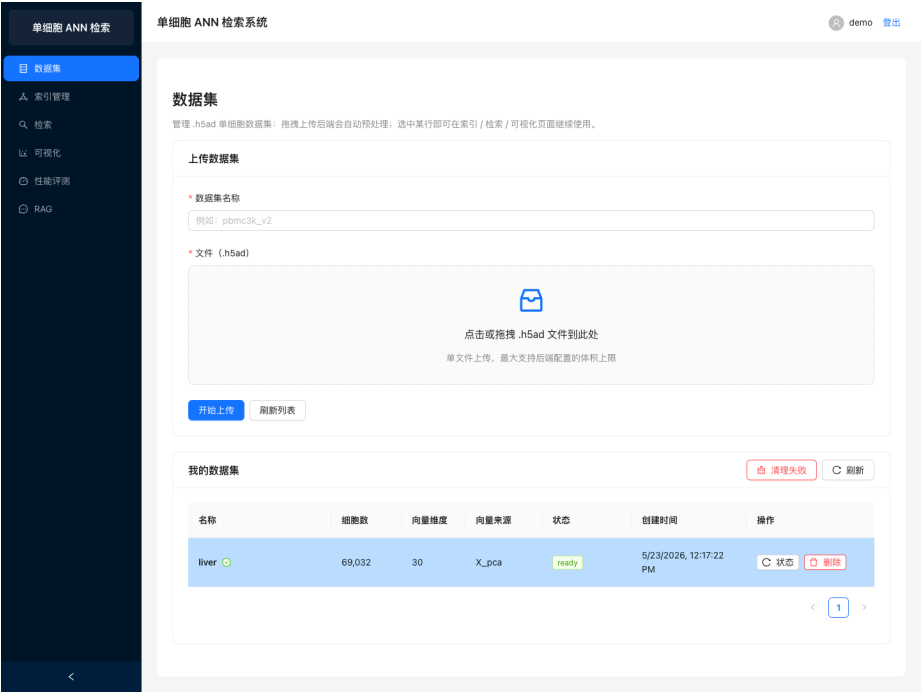
索引构建 (10 核 arm64, 单进程)

| backend       | 构建 (s)       | 内存 (MB)     | 关键参数            |
|---------------|--------------|-------------|-----------------|
| brute         | 0.000        | 3.43        | —               |
| hnswlib       | <b>0.224</b> | 7.10        | M=16, ef_c=200  |
| faiss-hnsw    | 0.245        | 3.43        | M=16, ef_c=200  |
| faiss-ivfpq   | 0.187        | <b>0.29</b> | nlist=173, m=10 |
| adaptive-hnsw | 0.218        | 7.10        | M=16, ef_c=200  |

单线程 Top-10 检索延迟与召回

| backend | p50 (ms)     | p95 (ms)     | p99 (ms) | QPS           | Recall@10     |
|---------|--------------|--------------|----------|---------------|---------------|
| brute   | 0.582        | 0.632        | 0.650    | 1 736         | 1.000         |
| hnswlib | <b>0.016</b> | <b>0.020</b> | 0.021    | <b>63 158</b> | <b>0.9996</b> |

6.1 演示截图集锦



# 6.1.1 相似细胞检索（真实数据）

检索

可视化

性能评测

RAG

按细胞 ID

按向量

多数据集联合

\* 数据集

liver (#3)

索引 (不选则用最新 ready)

自动选择

\* Top-K

10

\* 查询细胞 ID

CGACTTCTCCAGGGCT-1\_18

过滤器 (可选)

+ 添加过滤条件

发起检索

检索结果

耗时: 0.47 ms

返回数量: 10

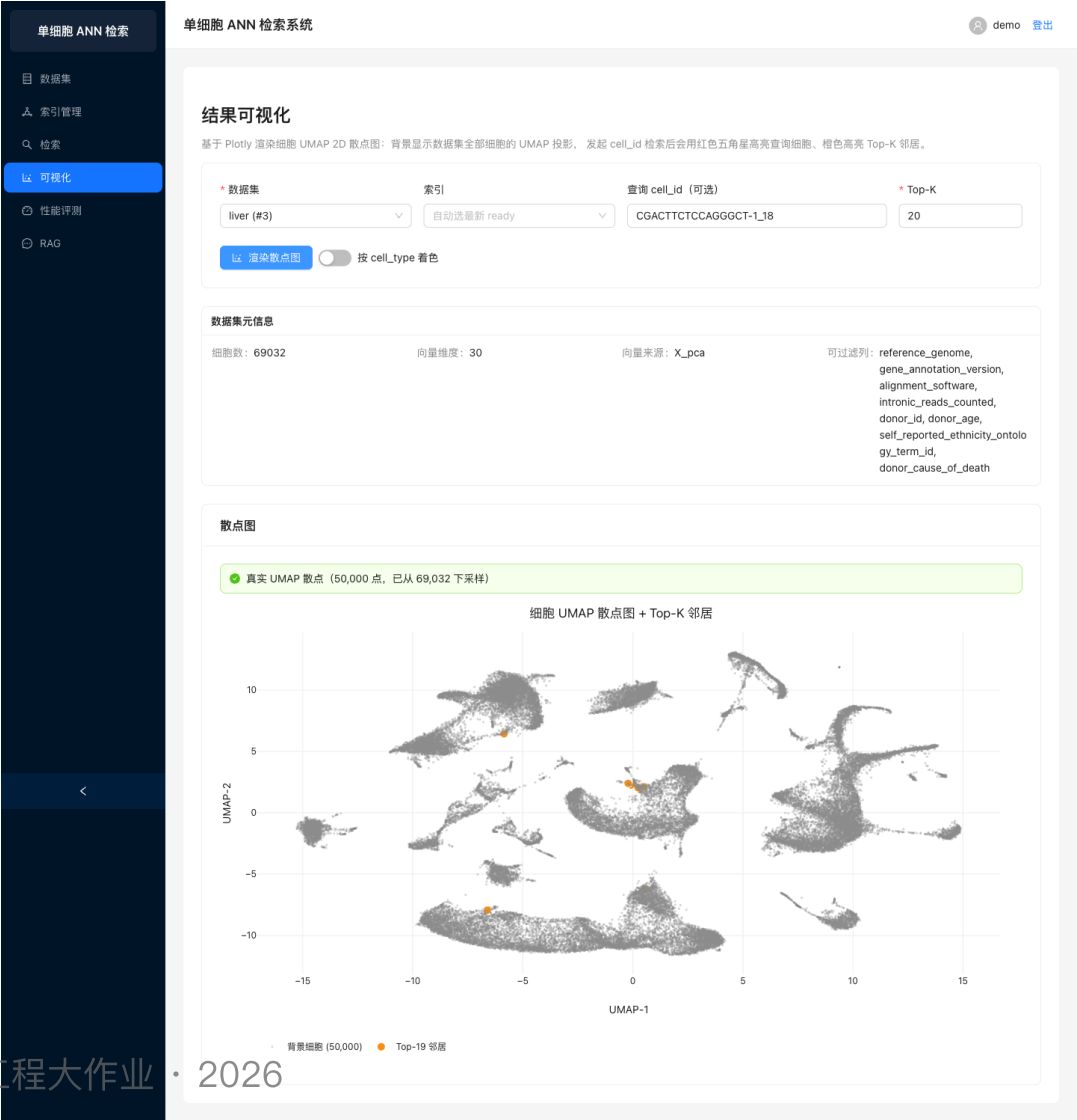
后端: hnsplib

距离: l2

| Rank | Cell ID                | Distance  | Metadata                                                                                                                                                                                                            |
|------|------------------------|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | TTGCCGTTCTCTTGAT-1_18  | 29.332634 | <div>cell_type: hepatocyte</div> <div>tissue: caudate lobe of liver</div> <div>disease: normal</div> <div>donor_age: &gt;60 years</div> <div>sex: female</div> <div>assay: 10x 3' v2</div> <div>+50 更多</div>        |
| 2    | GCACTCTAGCCCAACC-1_3   | 29.505344 | <div>cell_type: mononuclear phagocyte</div> <div>tissue: right lobe of liver</div> <div>disease: normal</div> <div>donor_age: 5-14 years</div> <div>sex: female</div> <div>assay: 10x 5' v2</div> <div>+50 更多</div> |
| 3    | GCGCAGTGTAACATCA-1_18  | 31.243437 | <div>cell_type: hepatocyte</div> <div>tissue: caudate lobe of liver</div> <div>disease: normal</div> <div>donor_age: &gt;60 years</div> <div>sex: female</div> <div>assay: 10x 3' v2</div> <div>+50 更多</div>        |
| 4    | ATCTACTCACAACGTT-1_18  | 31.941196 | <div>cell_type: hepatocyte</div> <div>tissue: caudate lobe of liver</div> <div>disease: normal</div> <div>donor_age: &gt;60 years</div> <div>sex: female</div> <div>assay: 10x 3' v2</div> <div>+50 更多</div>        |
| 5    | CAACCAACATTTTCAGG-1_18 | 32.027889 | <div>cell_type: hepatocyte</div> <div>tissue: caudate lobe of liver</div> <div>disease: normal</div> <div>donor_age: &gt;60 years</div> <div>sex: female</div> <div>assay: 10x 3' v2</div> <div>+50 更多</div>        |
| 6    | CCTAGCTTCCATTCTA-1_18  | 33.351643 | <div>cell_type: hepatocyte</div> <div>tissue: caudate lobe of liver</div> <div>disease: normal</div> <div>donor_age: &gt;60 years</div>                                                                             |



# 6.1.2 真实 UMAP 可视化（5 万点）



# 6.1.3 性能评测面板（实测）

单细胞 ANN 检索

数据集

索引管理

检索

可视化

性能评测

RAG

单细胞 ANN 检索系统

demo 登出

性能评测

对指定 ANN 索引执行 Recall / 延迟分位 / QPS 评测，结果异步落盘。 可前往 索引管理 选用索引以预填。

发起评测

\* 数据集

liver (#3)

\* 索引

#2 - hnswlib - 12

\* num\_queries

100

\* top\_k 列表

10 × 100 ×

\* 并发列表

1 × 4 × 8 × 16 ×

运行评测

刷新历史

历史评测结果

| 索引 ID | 数据集 | 后端      | Recall                 | 完成时间                   | 操作   |
|-------|-----|---------|------------------------|------------------------|------|
| 2     | #3  | hnswlib | R@10=99.9% R@100=99.6% | 5/23/2026, 10:29:49 PM | 查看详情 |

< 1 >

评测详情

构建耗时

681.8 ms

内存占用

16.33 MB

Recall@10

99.90 %

Recall@100

99.64 %

并发 vs 延迟 (ms)

| concurrency | P50   | P95   | P99   |
|-------------|-------|-------|-------|
| 1           | ~0.05 | ~0.05 | ~0.05 |
| 4           | ~0.08 | ~0.10 | ~0.35 |
| 8           | ~0.10 | ~0.18 | ~0.28 |
| 16          | ~0.12 | ~0.30 | ~0.55 |

并发 vs QPS

| concurrency | QPS  |
|-------------|------|
| 1           | ~58k |
| 4           | ~55k |
| 8           | ~62k |
| 16          | ~48k |

## 6.2 质量保障 Quality

| 维度   | 指标                                                                                |
|------|-----------------------------------------------------------------------------------|
| 后端测试 | 35 个 pytest 用例 (auth · datasets · ann · search · evaluation · rag · health), 全部通过 |
| 静态检查 | ruff lint + format · mypy 类型 · pre-commit 全绿                                      |
| 前端检查 | eslint + prettier · TypeScript strict 模式                                          |
| CI   | GitHub Actions 三步流水线: lint → test → build                                         |
| E2E  | Playwright 跑通真实数据全链路, 自动产出 9 张验收截图                                                |
| Git  | 25 次 约定式提交 ( feat: / fix: / docs: / chore: / build: / test: )                     |
| 文档   | 5 篇软件开发文档 ( 01_项目概述 ~ 05_用户手册 ) + 性能基准报告 + 答辩 PPT                                 |

### 关键提交节点:

- feat(backend): 自适应 HNSW 后端 + 基准测试脚本 (加分项)
- feat(backend): RAG 自然语言查询模块 (加分项)
- feat(backend): 检索与性能评测模块 (条件过滤 + 多数据集联合 + Recall/QPS/延迟)

# 6.3 项目交付清单

## 代码仓库 (GitHub)

- backend/ — FastAPI 后端 (22 个 Python 模块 · 35 测试)
- frontend/ — React 19 + TS 前端 (6 业务页面)
- infra/ — Docker Compose · Nginx · CI
- e2e/ — Playwright 端到端测试
- Makefile — 一键 make up / migrate / test / lint

## 软件开发文档 (docs/)

| 文件              | 内容                      |
|-----------------|-------------------------|
| 01_项目概述.md      | 背景 · 目标 · 计划            |
| 02_需求分析与系统设计.md | 用例 · 架构 · 接口 · 数据库 · UI |
| 03_系统测试.md      | 用例 · 性能测试               |
| 04_项目管理.md      | 分工 · 进度 · 工具            |

# 7.1 v1.1 性能优化与新功能总览

v1.0 → v1.1 共新增 26 个 commit · 覆盖 11 个 feat / perf 任务，全部已落地主分支并配套测试。

| 模块      | 关键优化                                                                            | 实测收益                                          |
|---------|---------------------------------------------------------------------------------|-----------------------------------------------|
| 缓存      | F4 IndexCache 启动预热 · F2 SearchCache (Redis SHA256 key)                          | 首查零冷启动； hit_ratio 实时可监控                       |
| 索引 / 存储 | F3 np.load(mmap_mode='r') · F5 VECTORS_DTYPE=float16 落盘                         | 冷启动 RSS -50% · 磁盘 -50% · Recall@10 仅降 0.4%    |
| 算法      | P2 numba @njit(parallel=True, fastmath=True) BruteBackend                       | 50k×50 数据 conc=1 p50 1.341 → 0.426 ms (3.15x) |
| API     | F1 /search/batch · F6 /search/by-vector-stream (SSE) · F8 Anthropic Claude Opus | 单数据集 N≤50 并发；首条结果即出即推；RAG 三客户端→四客户端           |
| DB      | P3 joinedload 消除 N+1 (datasets/index 列表 2→1 SQL)                                | 列表轮询 QPS 翻倍；SQL count 测试守住回归                  |
| 前端      | B1 plotly basic 瘦身 · B2 移动 Drawer · B3 Skeleton · D4 vitest hooks/store 单测      | 首屏 bundle 显著瘦身；移动端可用；前端单测覆盖率提升                |

统一节奏：每个 feat/perf 都在同一 commit 内带测试 + 文档同步更新 ( docs/06\_API接口文档.md 21 → 31 接口 )。

## 7.2 双层缓存: IndexCache + Redis SearchCache

### L1 · 进程内 IndexCache (LRU, capacity=4)

- F4 启动预热: lifespan 内查询 `status=ready` 最近 3 条索引一次性 `load`, 消除首查冷启动毛刺。
- `GET /indexes/cache/stats` 返回 `hits / misses / loads / evictions / hit_ratio / cached_index_ids`。
- 单进程内 `asyncio.Lock` 串行化同 `index_id` 并发加载, 避免重复 IO。

```
key = "search:" + sha256(  
    f"v1|{index_id}|{top_k}|{cid_or_vechash}|{filters_json}"  
).hexdigest()  
TTL = settings.SEARCH_CACHE_TTL_SECONDS # 默认 300s
```

```
请求 → IndexCache.get_or_load(index_id, db)  
    └─ hit → move_to_end(), 直接返回 backend 实例  
    └─ miss → create_backend + load(index_path)  
              插入 OrderedDict, 超容量 popitem(last=False)
```

### L2 · 跨进程 Redis SearchCache

- F2 接入 `by-id / by-vector / batch` 调用链, 命中走 `cache_hit=True` 直返。
- Redis 不可用 → 优雅降级为透传, 主链路不阻断 (`_errors` 计数器记录异常)。

## 7.3 算法 / 数据加速: mmap + float16 + numba

### F3 mmap 索引加载

- 大底库冷启动 RSS 减半，按需 page-in。
- hnswlib 0.8 load\_index 内部已切到 mmap 友好路径，与 F3 协同。

| backend | dtype | Recall@10     | p95(ms) | QPS(c=1) |
|---------|-------|---------------|---------|----------|
| brute   | fp32  | 1.0000        | 0.178   | 6 356    |
| brute   | fp16  | 1.0000        | 0.162   | 6 761    |
| hnswlib | fp32  | 0.9560        | 0.038   | 29 733   |
| hnswlib | fp16  | <b>0.9520</b> | 0.034   | 31 125   |

```
arr = np.load(path, mmap_mode="r", allow_pickle=False)
if arr.dtype == np.float32:
    self._vectors = arr      # 直接复用 memmap, 按页惰性读盘
else:
    self._vectors = arr.astype(np.float32, copy=True)  # float16 → float32 显式 astype
```

### F5 float16 落盘（消融，N=5000, d=50, l2)

→ 磁盘 / 冷启动内存 -50%，Recall@10 仅降 0.4%；  
与 F3 mmap 正交叠加。

# 7.4 检索 API 扩展矩阵

| #  | 接口                            | 关键能力                                                                                | 典型用途                                                |
|----|-------------------------------|-------------------------------------------------------------------------------------|-----------------------------------------------------|
| F1 | POST /search/batch            | queries: [{cell_id, vector}] · 1 ≤ N ≤ 50 ·<br>asyncio.gather 并发 · 每条独立 SearchCache | 评测脚本一次性提交, 全局<br>total_latency_ms + 逐条<br>cache_hit |
| F6 | POST /search/by-vector-stream | text/event-stream SSE · event: hit 每 ~20ms<br>推一条 · 末尾 event: done 汇总 latency_ms    | 前端"逐条飞入"动画, 消除等待感                                   |
| F7 | POST /search/multi-dataset    | 并发查多数据集 · min-max 归一化 · 全局 Top-K · 标<br>source_dataset_id                           | 跨数据集 ensemble (v1.0 加分项 ①)                          |
| F8 | LLM_PROVIDER=anthropic        | AnthropicClient 接入 Claude Opus 4 · 工厂分支 +<br>失败自动 fallback Mock                     | RAG parse_query / summarize 第四<br>种 LLM             |

## F1 batch 请求示例

```
POST /api/v1/search/batch
{
  "dataset_id": 1, "index_id": 7, "top_k": 10,
  "filters": {"cell_type": "hepatocyte"},
  "queries": [
    {"cell_id": "AAACCTGAGAATCT-1"},
    {"vector": [0.12, -0.03, ...]}
```



# 总结 Conclusion

一个完整的端到端 Web 系统

注册 → 上传 → 预处理 → 多后端索引 → 条件检索 → 可视化 → RAG 问答

三个加分项全部落地

多数据集联合 · 自适应 HNSW · RAG 自然语言

实测可量产的性能

微秒级延迟 · 99.96% Recall@10 · 6 万 QPS

谢谢聆听 · 欢迎提问

Q & A